```
#Name: Chaitanya Kharpate
#PRN: 23070521037
#Section: A
```

```
import kagglehub

# Download latest version
path = kagglehub.dataset_download("ashydv/housing-dataset")

print("Path to dataset files:", path)
```

```
Downloading to /root/.cache/kagglehub/datasets/ashydv/housing-dataset/1.archive...
100%|██████████| 4.63k/4.63k [00:00<00:00, 1.16MB/s]Extracting files...
Path to dataset files: /root/.cache/kagglehub/datasets/ashydv/housing-dataset/versions/1
```

```
#PRN: 23070521075
#NAME: Abhinava Akash Kotha
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.metrics import mean_absolute_error, mean_squared_error

import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Input
from tensorflow.keras.callbacks import EarlyStopping
```

```
import os

# Load dataset
df = pd.read_csv(os.path.join(path, "Housing.csv"))

print(df.head())
print("Shape:", df.shape)
print("Columns:", list(df.columns))
```

```
      price  area  bedrooms  bathrooms  stories mainroad guestroom basement  \
0  13300000  7420         4          2        3      yes        no       no
1  12250000  8960         4          4        4      yes        no       no
2  12250000  9960         3          2        2      yes        no      yes
3  12215000  7500         4          2        2      yes        no      yes
4  11410000  7420         4          1        2      yes       yes      yes

  hotwaterheating airconditioning  parking prefarea furnishingstatus
0              no             yes        2      yes        furnished
1              no             yes        3       no        furnished
2              no              no        2      yes   semi-furnished
3              no             yes        3      yes        furnished
4              no             yes        2       no        furnished
Shape: (545, 13)
Columns: ['price', 'area', 'bedrooms', 'bathrooms', 'stories', 'mainroad', 'guestroom', 'basement', 'hotwaterheating', 'airc
```

```
# Features and Target
X = df.drop(columns=["price"])
y = df["price"]

# Identify column types
categorical_cols = X.select_dtypes(include=["object"]).columns.tolist()
numerical_cols = X.select_dtypes(exclude=["object"]).columns.tolist()

# Numerical preprocessing
numeric_transformer = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])

# Categorical preprocessing
categorical_transformer = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
```

```
    ("onehot", OneHotEncoder(handle_unknown="ignore"))
])

# Combine preprocessing
preprocessor = ColumnTransformer([
    ("num", numeric_transformer, numerical_cols),
    ("cat", categorical_transformer, categorical_cols)
])
```

```
# Train Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

# Transform data
X_train = preprocessor.fit_transform(X_train)
X_test = preprocessor.transform(X_test)

print("Training Shape:", X_train.shape)
print("Testing Shape:", X_test.shape)
```

```
Training Shape: (436, 20)
Testing Shape: (109, 20)
```

```
# Build Neural Network
model = Sequential([
    Input(shape=(X_train.shape[1],)),
    Dense(128, activation="relu"),
    Dense(64, activation="relu"),
    Dense(32, activation="relu"),
    Dense(1)
])

# Compile
model.compile(
    optimizer="adam",
    loss="mean_squared_error"
)

# Early stopping
early_stop = EarlyStopping(
    monitor="val_loss",
    patience=10,
    restore_best_weights=True
)
```

```
# Train
history = model.fit(
    X_train,
    y_train,
    validation_split=0.2,
    epochs=200,
    batch_size=32,
    callbacks=[early_stop],
    verbose=1
)

# Predict
pred = model.predict(X_test).ravel()

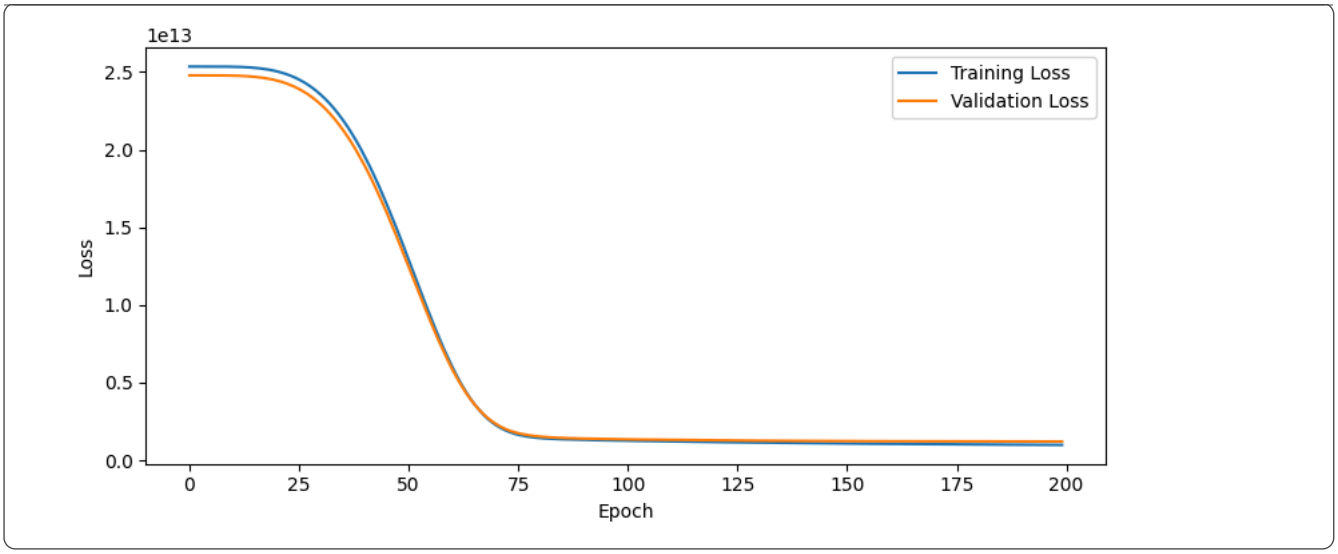
# Evaluation
mae = mean_absolute_error(y_test, pred)
mse = mean_squared_error(y_test, pred)
rmse = np.sqrt(mse)

print("MAE:", round(mae, 2))
print("MSE:", round(mse, 2))
print("RMSE:", round(rmse, 2))
```

```
Epoch 1/200
11/11 ━━━━━━━━━━━━━━━━━━━━ 3s 52ms/step - loss: 25349379325952.0000 - val_loss: 24781577519104.0000
```

```
Epoch 2/200
11/11 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - loss: 25349354160128.0000 - val_loss: 24781543964672.0000
Epoch 3/200
11/11 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - loss: 25349301731328.0000 - val_loss: 24781470564352.0000
Epoch 4/200
11/11 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 25349192679424.0000 - val_loss: 24781311180800.0000
Epoch 5/200
11/11 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 25348964089856.0000 - val_loss: 24780996608000.0000
Epoch 6/200
11/11 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - loss: 25348511105024.0000 - val_loss: 24780409405440.0000
Epoch 7/200
11/11 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - loss: 25347709992960.0000 - val_loss: 24779369218048.0000
Epoch 8/200
11/11 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - loss: 25346325872640.0000 - val_loss: 24777634873344.0000
Epoch 9/200
11/11 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - loss: 25344079822848.0000 - val_loss: 24774879215616.0000
Epoch 10/200
11/11 ━━━━━━━━━━━━━━━━━━━━ 0s 26ms/step - loss: 25340562898944.0000 - val_loss: 24770731048960.0000
Epoch 11/200
11/11 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - loss: 25335349379072.0000 - val_loss: 24764695445504.0000
Epoch 12/200
11/11 ━━━━━━━━━━━━━━━━━━━━ 0s 35ms/step - loss: 25327965306880.0000 - val_loss: 24756233437184.0000
Epoch 13/200
11/11 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - loss: 25317565530112.0000 - val_loss: 24744772501504.0000
Epoch 14/200
11/11 ━━━━━━━━━━━━━━━━━━━━ 1s 30ms/step - loss: 25303955013632.0000 - val_loss: 24729457000448.0000
Epoch 15/200
11/11 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - loss: 25285995003904.0000 - val_loss: 24709559222272.0000
Epoch 16/200
11/11 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - loss: 25263039578112.0000 - val_loss: 24684315803648.0000
Epoch 17/200
11/11 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - loss: 25233757044736.0000 - val_loss: 24652862717952.0000
Epoch 18/200
11/11 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - loss: 25197849608192.0000 - val_loss: 24614094766080.0000
Epoch 19/200
11/11 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 25153251573760.0000 - val_loss: 24567147921408.0000
Epoch 20/200
11/11 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 25099163926528.0000 - val_loss: 24510828904448.0000
Epoch 21/200
11/11 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 25035314036736.0000 - val_loss: 24443338358784.0000
Epoch 22/200
11/11 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 24959657181184.0000 - val_loss: 24363451547648.0000
Epoch 23/200
11/11 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 24869718720512.0000 - val_loss: 24271241871360.0000
Epoch 24/200
11/11 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 24767226707968.0000 - val_loss: 24163836231680.0000
Epoch 25/200
11/11 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 24647017955328.0000 - val_loss: 24041423372288.0000
Epoch 26/200
11/11 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 24510631772160.0000 - val_loss: 23901574791168.0000
Epoch 27/200
11/11 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - loss: 24356602249216.0000 - val_loss: 23742593892352.0000
Epoch 28/200
11/11 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 24180084965376.0000 - val_loss: 23564799442944.0000
Epoch 29/200
11/11 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 23984735256576.0000 - val_loss: 23364653547520.0000
```

```
# Plot Loss
plt.figure(figsize=(8,4))
plt.plot(history.history["loss"], label="Training Loss")
plt.plot(history.history["val_loss"], label="Validation Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.tight_layout()
plt.show()
```


1e13
2.5
2.0
1.5
1.0
0.5
0.0
Loss
0
25
50
75
100
125
150
175
200
Epoch
Training Loss
Validation Loss